

Code-Layout, Readability and Reusability

Date	Team ID	Project Name	Max Marks
05 July 2026	*****	StudyAI – AI Powered Smart Study Companion	5 Marks

S.No	Quality Principle	Implementation Details / Description	Specific Project Examples (Files/Folders)
1	Folder/Directory Structure	Frontend organized by role (pages, components, services); backend follows a routes–services separation so HTTP handling is decoupled from business logic	Frontend: /pages, /components, /services; Backend: /routes, /services, /uploads
2	Naming Conventions	camelCase for JS variables/functions, PascalCase for React components, snake_case for Python files/functions	groq_service.py, pdf_service.py (Python); DashboardCard.jsx, api.js (React)
3	Code Readability & Comments	Backend service functions documented with docstrings explaining input/output; complex extraction/parsing logic commented inline	Docstrings in pdf_service.py explaining extraction logic; inline comments in groq_service.py for API payload structure
4	Modularity & Reusability	Shared UI elements (Navbar, Sidebar) built once and reused across all 7 pages; backend split into route handlers vs. service logic so summary/flashcard/quiz generation can reuse the same Groq service function	Navbar and Sidebar components reused across Home, Upload, Summary, Flashcards, Quiz, Planner, Dashboard; groq_service.py reused by summary, flashcard, and quiz generation logic
5	Formatting & Linting Tools	<i>[State honestly — if you haven't set up ESLint/Prettier yet, say so rather than claiming it]</i>	<i>[e.g., "Not yet configured" or "ESLint with default CRA config" if applicable]</i>
6	Error Handling Patterns	Flask routes wrapped in try/except blocks returning consistent JSON error responses; frontend displays error state if API call fails	Try/except in upload_routes.py and summary_routes.py; error state handling in api.js calls